5.API Design Document – AI-Based Smart Quiz System

5.1 Introduction

This document describes the API design for the AI-Based Smart Quiz System, a Laravel-based web application supporting two user roles: Teacher (is_teacher = 1) and Student (is_teacher = 0). The system uses Laravel‟s session-based authentication rather than token-based mechanisms. All routes are standard Laravel web routes protected by the „auth‟ and role-checking middleware. There are no Bearer tokens or API keys passed in request headers from the client side; instead, the authenticated session cookie is used to identify and authorize users. The APIs are grouped into the following categories: Authentication, Teacher – Course Management, Teacher – Question Management, AI Question Generation, Student Quiz, Leaderboard, Suspicious Attempt Monitoring, and User Profile.

5.2 Authentication APIs

Method	Endpoint	Purpose	Input Data	Expected Output
POST	/register	Register a new user (Teacher or Student)	name, email, password, password_confirmation, is_teacher (0 or 1)	Redirects to dashboard on success; returns validation errors on failure
POST	/login	Authenticate an existing user and start a session	email, password, remember (optional boolean)	Redirects to role-based dashboard on success; returns error message on failure
POST	/logout	Terminate the authenticated user session	None (uses active session token via CSRF)	Redirects to login page; session is invalidated

5.3 Teacher – Course Management APIs

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/teacher/courses	Retrieve and display all courses belonging to the authenticated teacher	None	HTML view listing all courses with Edit and Delete options
POST	/teacher/courses	Create a new course	title (string), description (text), timer_minutes (integer – configurable quiz timer)	Redirects to course list on success; validation errors returned on failure
GET	/teacher/courses/{id}/edit	Show the edit form pre-populated with the selected course‟s data	URL parameter: id (course ID)	HTML edit form with existing course data populated
PUT	/teacher/courses/{id}	Update an existing course‟s details	URL parameter: id; Body: title, description, timer_minutes	Redirects to course list on success; validation errors on failure

Method	Endpoint	Purpose	Input Data	Expected Output
DELETE	/teacher/courses/{id}	Permanently delete a course and its associated questions	URL parameter: id (course ID)	Redirects to course list with a success message

5.4 Teacher – Question Management (Manual) APIs

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/teacher/courses/{course_id}/questions	List all MCQ questions associated with a specific course	URL parameter: course_id	HTML view listing all questions with Edit and Delete options

POST	/teacher/courses/{course_id}/questions	Add a new MCQ question to a specific course	URL parameter: course_id; Body: question_text, option_a, option_b, option_c, option_d, correct_answer	Redirects to question list on success; validation errors on failure
Method	**Endpoint**	**Purpose**	**Input Data**	**Expected Output**
			(a/b/c/d), explanation	
GET	/teacher/questions/{id}/edit	Display the edit form for a specific question	URL parameter: id (question ID)	HTML edit form pre-filled with the question‟s data
PUT	/teacher/questions/{id}	Update the content of an existing MCQ question	URL parameter: id; Body: question_text, option_a, option_b, option_c, option_d, correct_answer	Redirects to question list on success; validation errors on failure

			, explanation	
DELETE	/teacher/questions/{id}	Delete a specific MCQ question permanently	URL parameter: id (question ID)	Redirects to question list with a success confirmation message

5.5 AI Question Generation API

The teacher enters a topic, and the server calls the Google Gemini API to generate a complete MCQ. The result is returned as JSON so the teacher can review and save it.

Method	Endpoint	Purpose	Input Data	Expected Output
POST	/teacher/generate-ai-question	Call the Google Gemini API with a given topic and return a generated MCQ in JSON format	Body: topic (string – the subject for which the question should be generated), course_id (to verify it is Mobile Computing Lab)	JSON response: { "question": "...", "options": ["A. ...", "B. ...", "C. ...", "D. "], "correct_answer": "A", "explanation": " " }

5.6 Student Quiz APIs

These routes allow students to load and take timed quizzes. For the “Mobile Computing Lab” course, the quiz page embeds hidden fields to capture cheating detection data. The timer is configurable per course (set by the teacher). Upon submission, the server evaluates answers, computes average time per question, and flags the attempt as suspicious if warranted.

Method	Endpoint	Purpose	Input Data	Expected Output

GET	/quiz/{course_id}	Load the quiz page for a specific course, including questions, timer	URL parameter: course_id	HTML quiz page with: all MCQ questions, a countdown timer (from timer_minutes), hidden fields: tab_switch_count
Method	**Endpoint**	**Purpose**	**Input Data**	**Expected Output**
		configuration, and hidden fields for cheating detection		(default 0, incremented by JavaScript visibilitychange event) and quiz_start_time (Unix timestamp in milliseconds)

POST	/quiz/submit	Submit quiz answers. Server calculates score, average time per question, and determines whether the attempt is suspicious	Body: course_id, answers[] (array of selected options per question), tab_switch_count (integer), quiz_start_time (integer, milliseconds). Cheating rule: attempt is flagged as suspicious if tab_switch_count > 0 OR avg_time_per_question < 5 seconds	Saves QuizAttempt record with score, time_taken, tab_switch_count, avg_time_per_question, is_suspicious flag; Redirects to /result/{attempt_id}
GET	/result/{attempt_id}	Display the result page for a completed	URL parameter: attempt_id	HTML result page showing: total score, per-question breakdown (student‟s
Method	**Endpoint**	**Purpose**	**Input Data**	**Expected Output**
		quiz attempt		answer vs. correct answer), explanations for each question, and total time taken

5.7 Leaderboard API

The leaderboard ranks students by score (descending) and time taken (ascending) for a given course. Attempts flagged as suspicious (is_suspicious = 1) are excluded from the ranking to ensure integrity.

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/leaderboard/{course_id}	Display the ranked leaderboard for a specific course, excluding suspicious attempts	URL parameter: course_id	HTML leaderboard view showing: Rank, Student Name, Score, Time Taken; ordered by score DESC and time_taken ASC; all records where is_suspicious = 0

5.8 Teacher – Suspicious Attempts Monitoring API

This endpoint allows teachers to review all quiz attempts flagged as suspicious for the “Mobile Computing Lab” course. An attempt is marked suspicious when tab_switch_count > 0 OR avg_time_per_question < 5 seconds.

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/teacher/suspicious-attempts	List all flagged quiz attempts for the "Mobile Computing Lab" course for teacher review	None (teacher-authenticated session required)	HTML view listing all suspicious attempts with: Student Name, Score, tab_switch_count, avg_time_per_question (in seconds), attempted_at (datetime)

5.9 User Profile APIs

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/profile	Display the authenticated user‟s profile information	None (uses active session)	HTML profile page showing: name, email, role (Teacher or Student)
PUT	/profile	Update the authenticated user‟s profile details	Body: name (string), email (string), password (string, optional), password_confirmation (required if password provided)	Redirects to profile page with a success message; validation errors returned on failure

Note on Cheating Detection Data Transmission

Cheating detection data for the "Mobile Computing Lab" course is transmitted via hidden HTML form fields embedded in the quiz page. When the quiz page loads (GET /quiz/{course_id}), two

hidden input fields are rendered in the form: (1) quiz_start_time – populated by JavaScript with Date.now() at the moment the student begins the quiz, recorded in milliseconds; and (2) tab_switch_count – initialized to 0 and incremented in real time by a JavaScript event listener on the document.visibilitychange event each time the student switches away from the quiz tab. When the student submits the quiz (POST /quiz/submit), both values are posted along with the answers array and course_id as standard HTML form fields. The server then computes avg_time_per_question = (submission_time – quiz_start_time) / 1000 / total_questions, and applies the cheating detection rule: if tab_switch_count > 0 OR avg_time_per_question < 5, the attempt is saved with is_suspicious = 1 and excluded from the leaderboard.